# bok25
# 基礎班、考研数据結構

# 红黑树

# 红黑树

定义：符合以下要求的树：

1.节点是红色或黑色

2.根节点是黑色

3.所有叶子结点都是黑色（叶子是NIL节点）

4.每个红色节点必须有两个黑色的子节点。
（或者说从每个叶子到根的所有路径上不能有两个连续的红色节点。）
（或者说不存在两个相邻的红色节点，相邻指两个节点是父子关系。）
（或者说红色节点的父节点和子节点均是黑色的。）

5.从任一节点到其每个叶子的所有路径都包含相同数目的黑色节点

# 红黑树

定义：符合以下要求的树：

1.节点是红色或黑色

2.根节点是黑色

3.所有叶子结点都是黑色（叶子是NIL节点）

4.每个红色节点必须有两个黑色的子节点。
（或者说从每个叶子到根的所有路径上不能有两个连续的红色节点。）
（或者说不存在两个相邻的红色节点，相邻指两个节点是父子关系。）
（或者说红色节点的父节点和子节点均是黑色的。）

5.从任一节点到其每个叶子的所有路径都包含相同数目的黑色节点

名词解释：

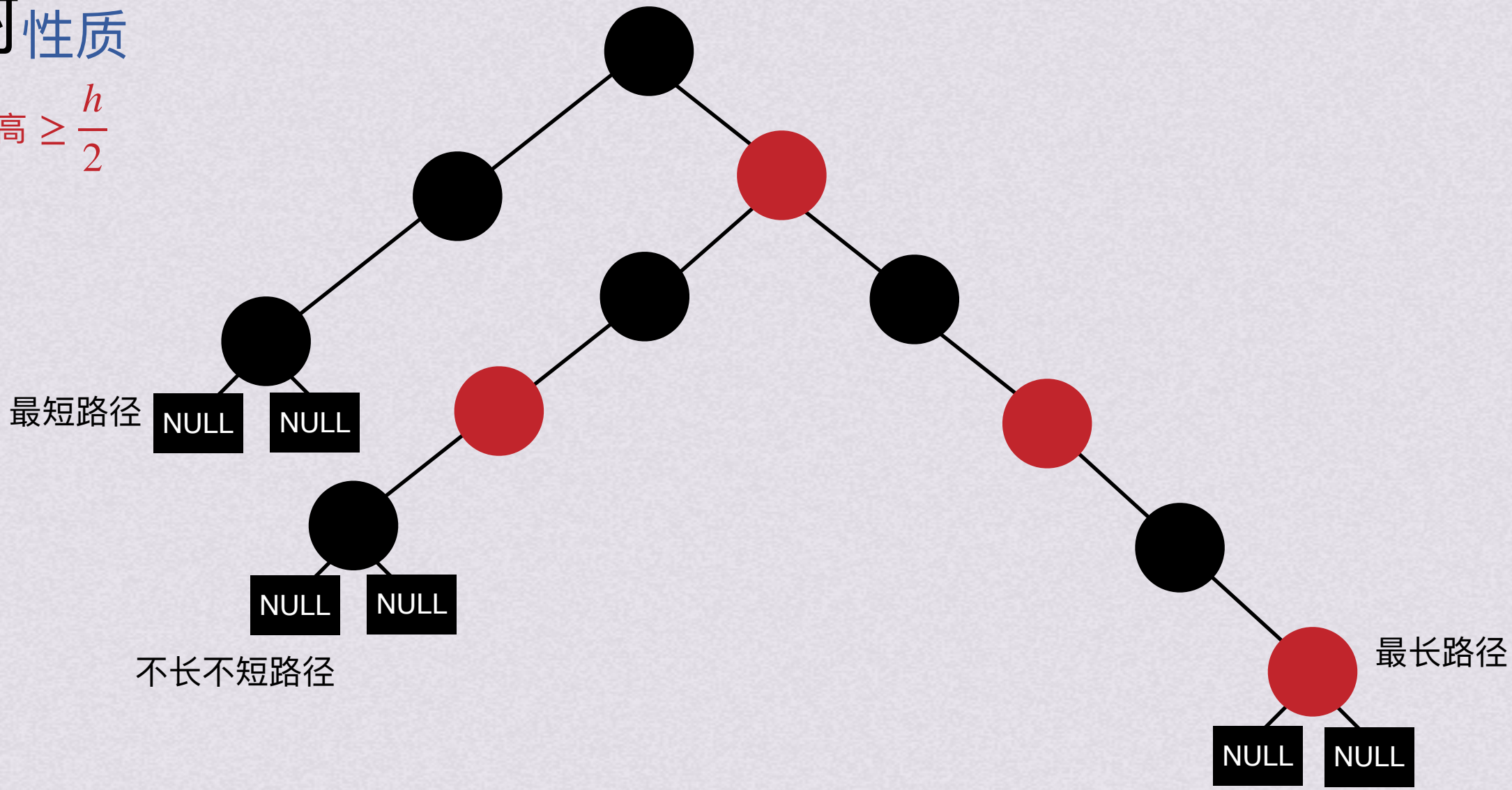
**黑高（bh）**：从某节点出发（不含该节点）到达任一叶节点（包含叶节点）的路径上的黑节点总数

**内部节点**：红黑树的非叶节点（包括根节点）

**外部节点**：红黑树的叶子节点

# 红黑树性质

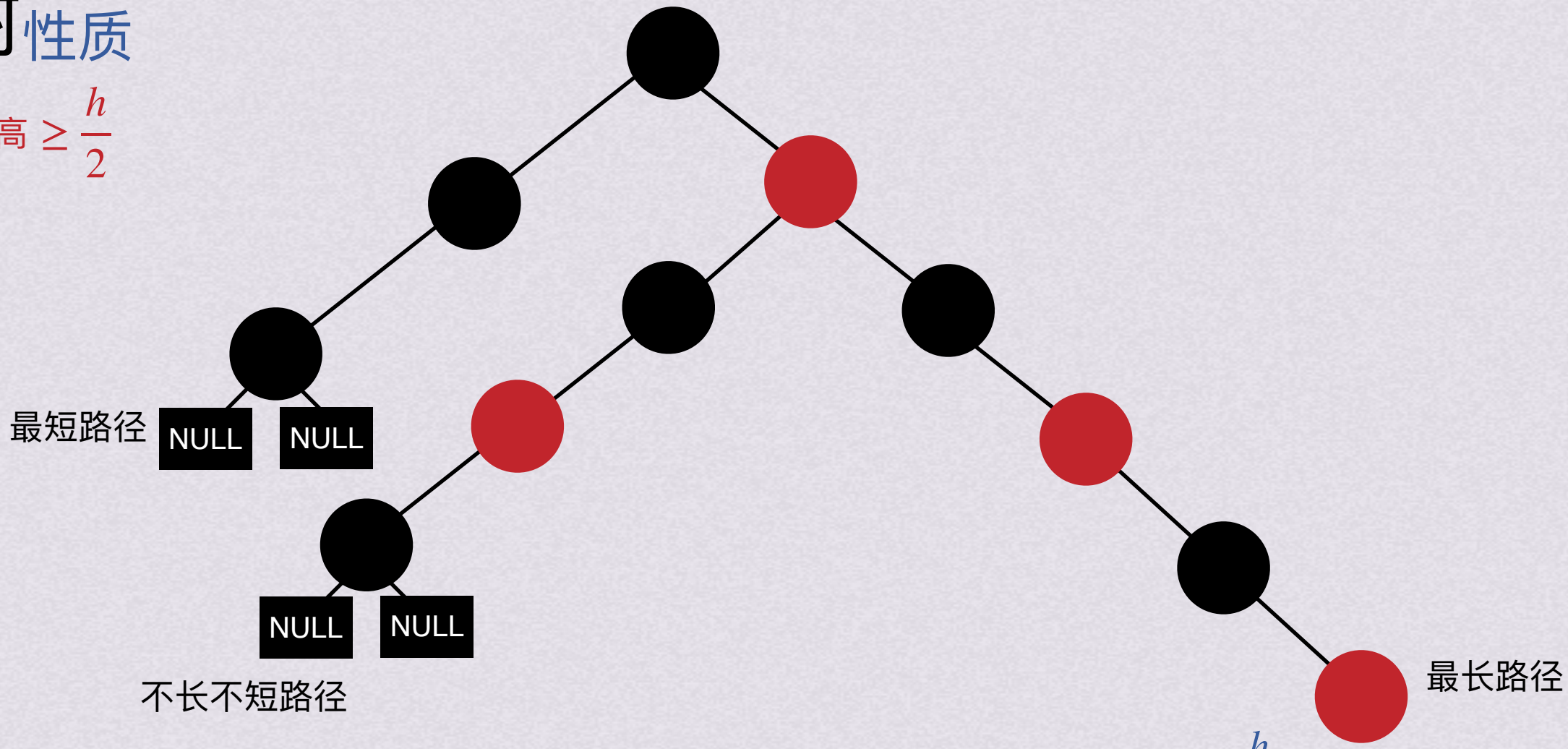
- 根节点的黑高 $\geq \frac{h}{2}$




**外部节点**：红黑树的叶子节点

# 红黑树性质

- 根节点的黑高 $\geq \frac{h}{2}$

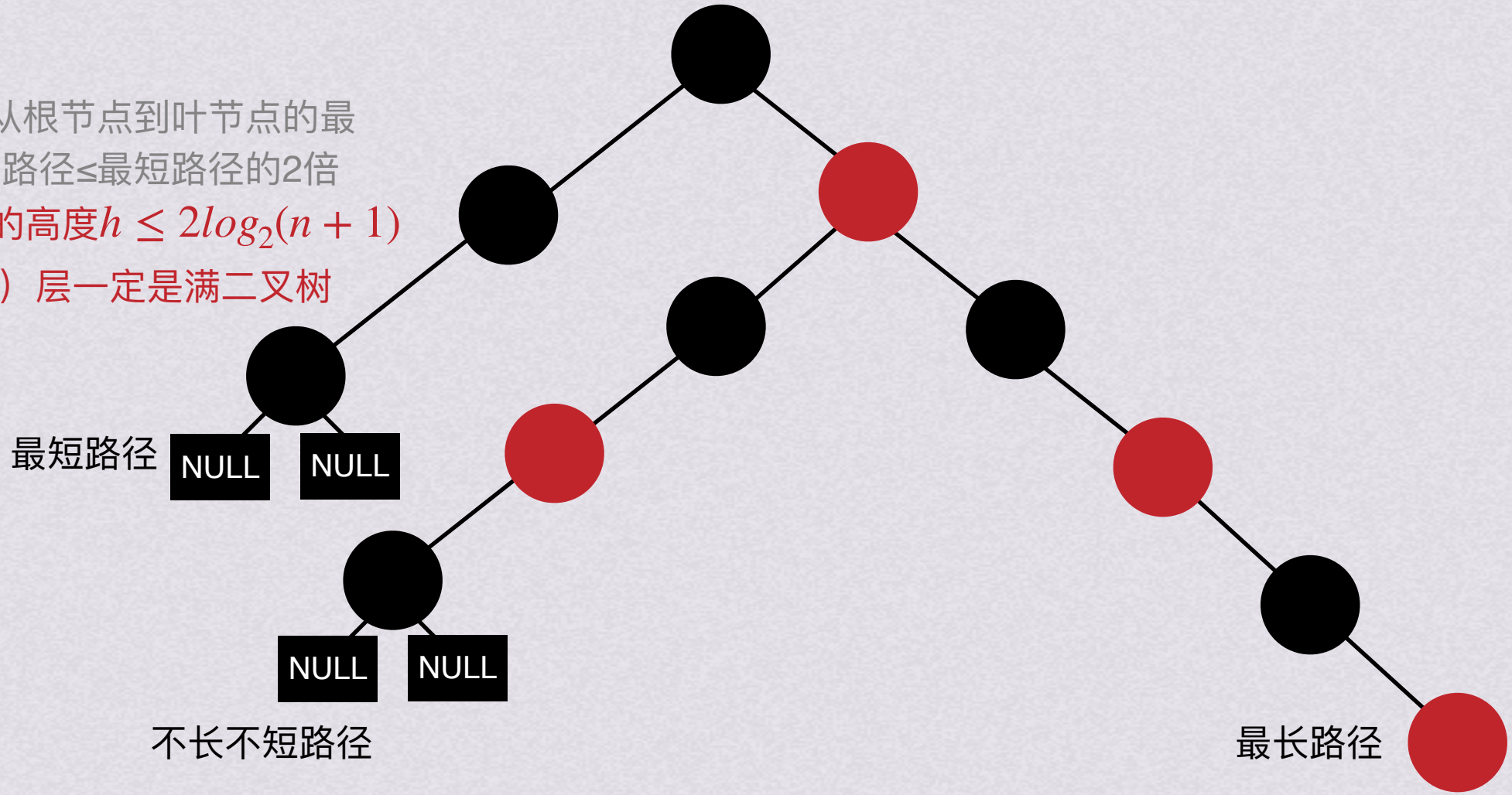



红节点的个数和黑节点的个数相同 = $\frac{h}{2}$

$\frac{h}{2}$ 就是这棵树中任意一条路径中红节点数量能达到的最大值

# 红黑树性质

- 根节点的黑高 $\geq \frac{h}{2}$ //从根节点到叶节点的最长路径≤最短路径的2倍
- 有n个内部节点的红黑树的高度$h \leq 2log_2(n+1)$
- 树的前bh（黑高）（root）层一定是满二叉树
- $n \geq 2^{\frac{h}{2}} - 1$

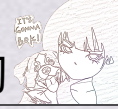



红节点的个数和黑节点的个数相同 = $\frac{h}{2}$

$\frac{h}{2}$ 就是这棵树中任意一条路径中红节点数量能达到的最大值

# 红黑树

## 插入



# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

# 红黑树插入

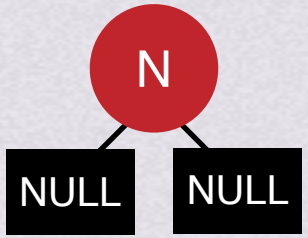
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 1.插入空树

# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

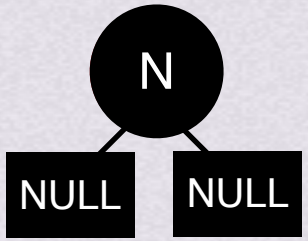
## 1.插入空树



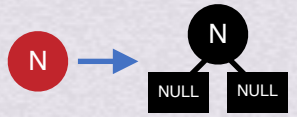
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 1.插入空树



# 红黑树插入

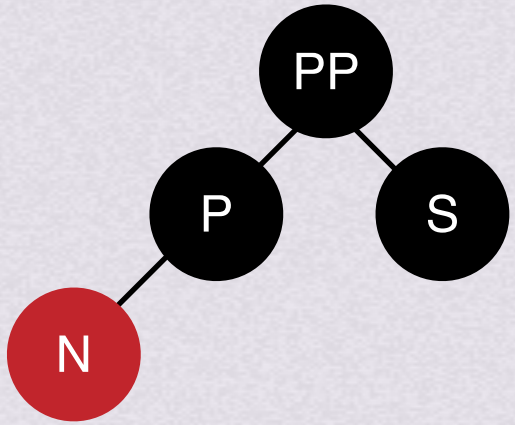



1.插入空树

直接插入，染色

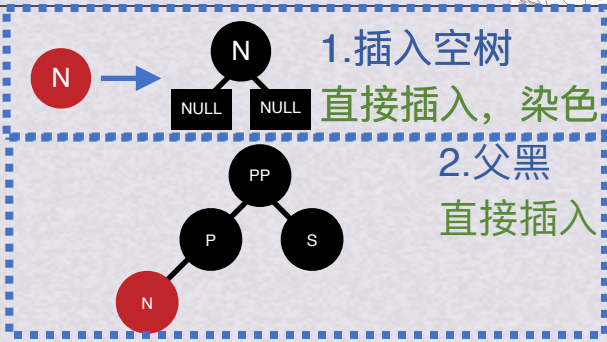
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 2.父黑



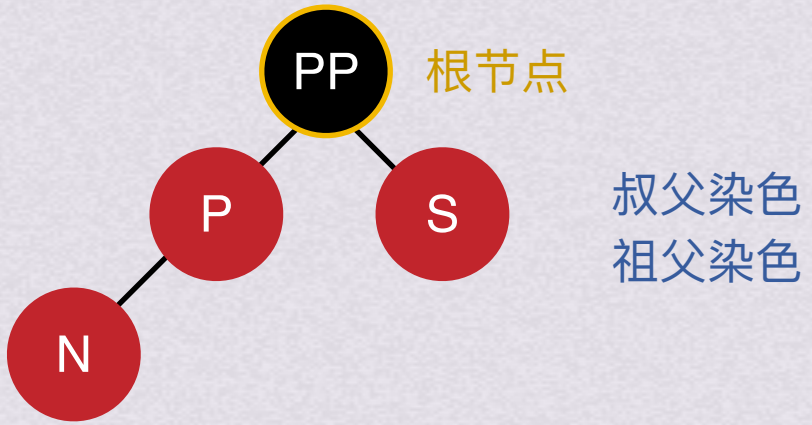
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!



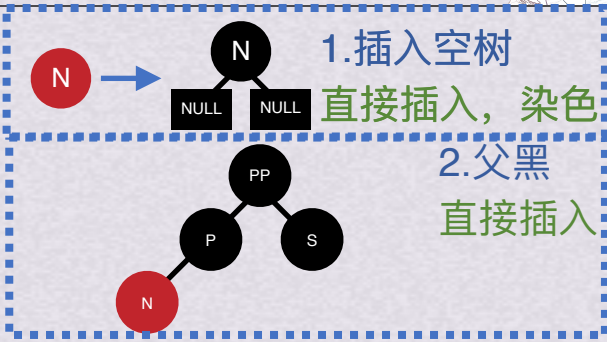

## 3.父红且叔红

## 3-1祖父是根节点



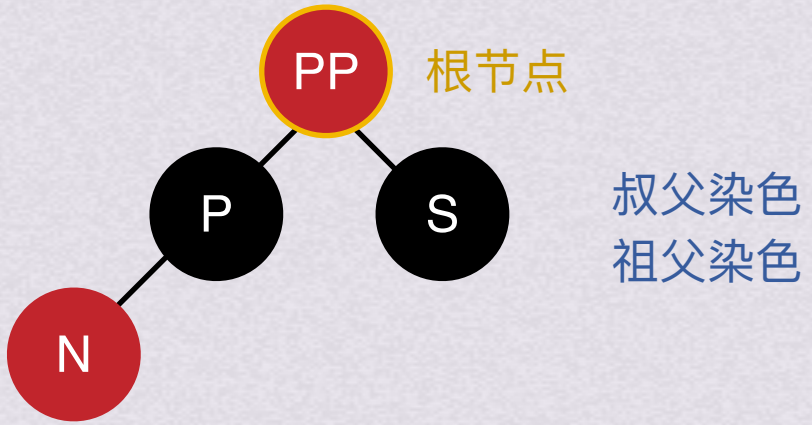
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!



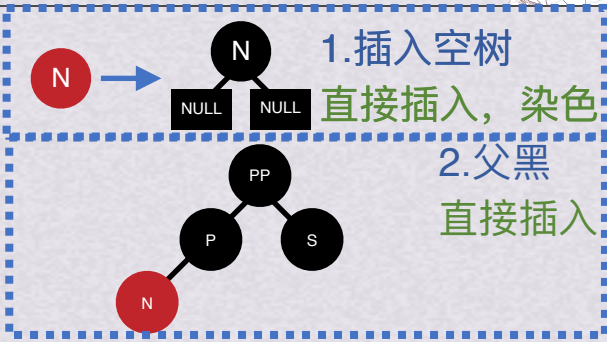

## 3.父红且叔红

## 3-1祖父是根节点



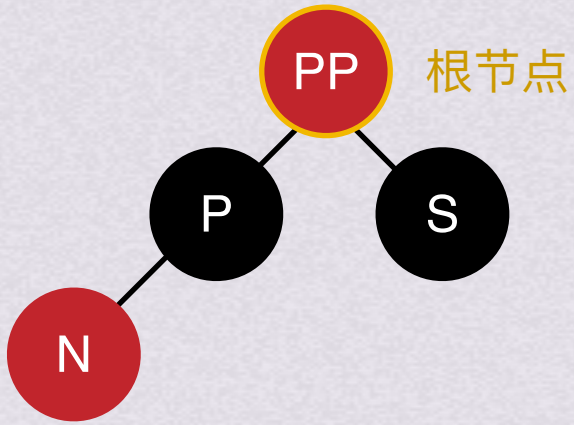
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!




## 3.父红且叔红

## 3-1祖父是根节点



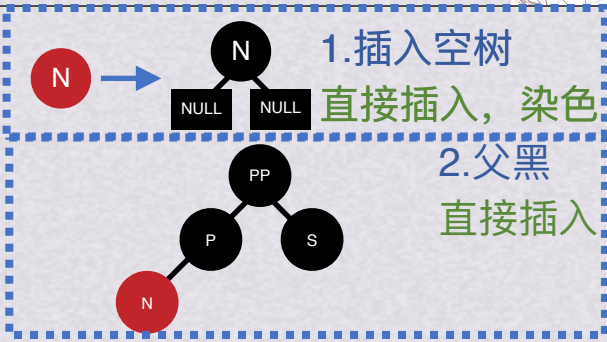

根节点

祖父染色

叔父染色
祖父染色

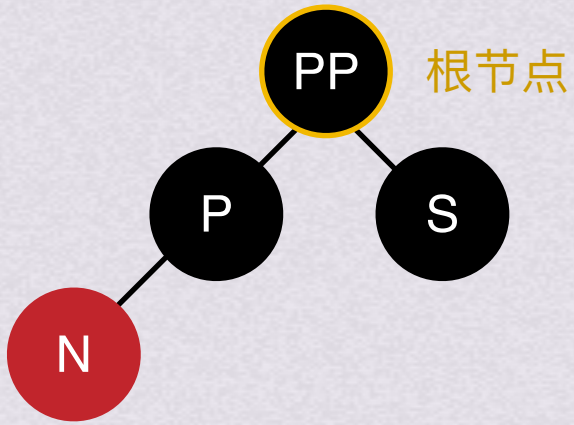
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!




## 3.父红且叔红

## 3-1祖父是根节点




根节点

祖父染色

叔父染色
祖父染色

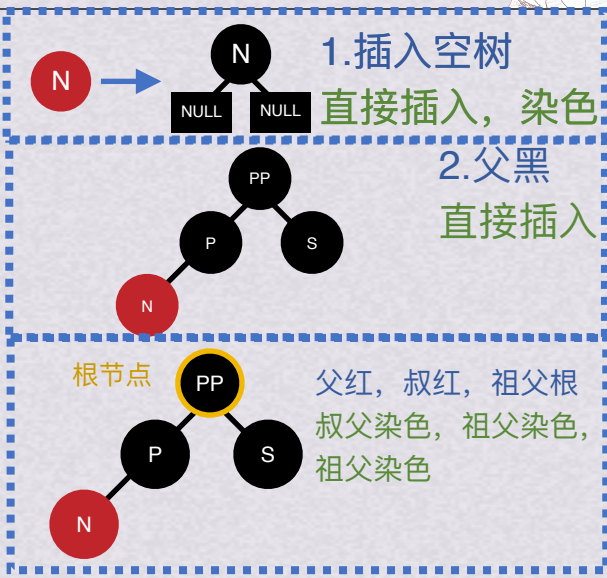
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 3.父红且叔红

## 3-2祖父不是根节点

祖父成为新节点，继续向上调整

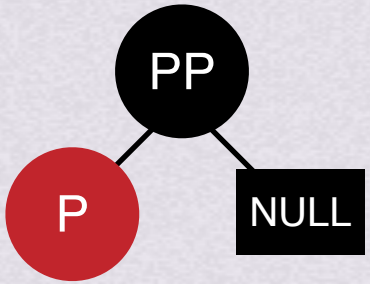


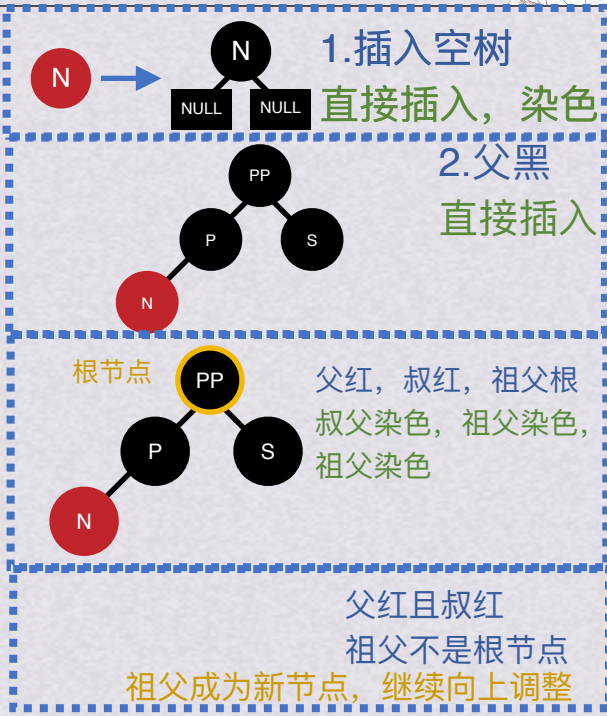
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的左孩子

插入节点为其父的左孩子



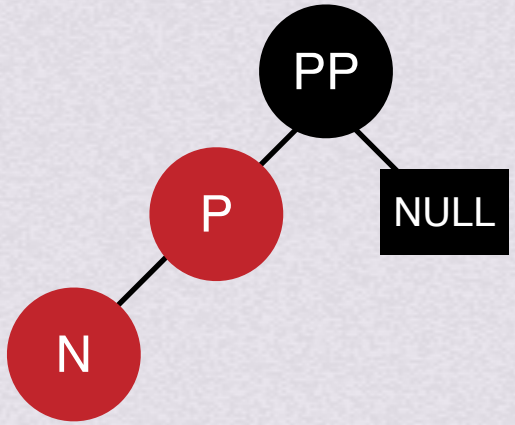



# 红黑树插入

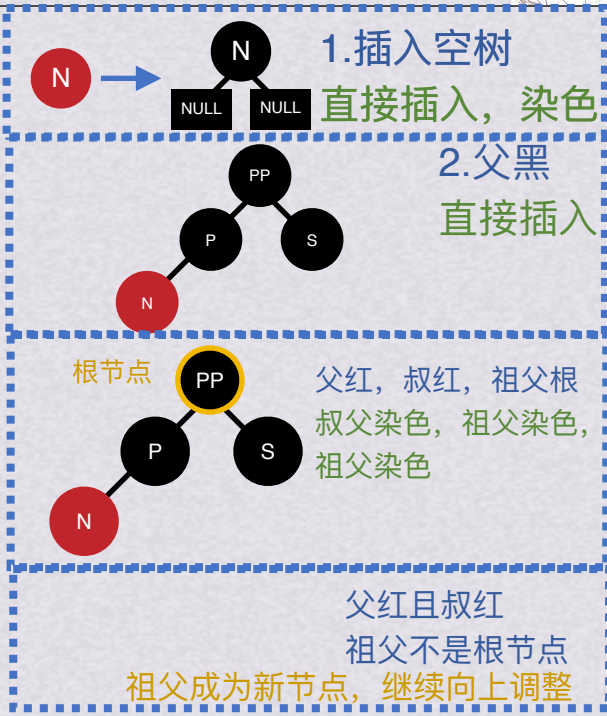
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的左孩子

插入节点为其父的左孩子



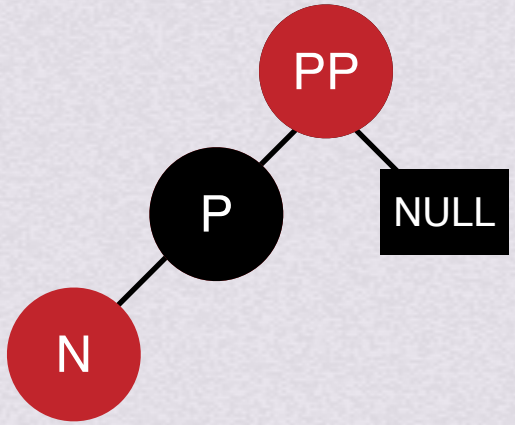

父亲、祖父染色



# 红黑树插入

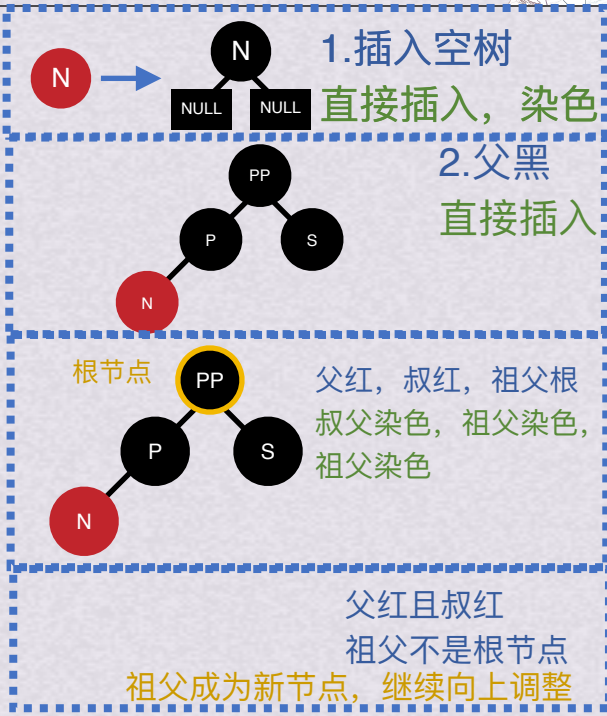
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的左孩子
插入节点为其父的左孩子




然后右旋

父亲、祖父染色

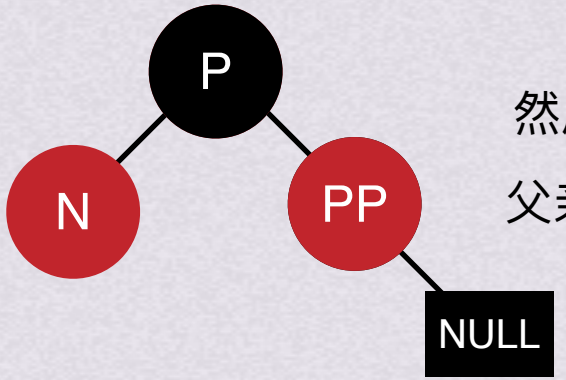


# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

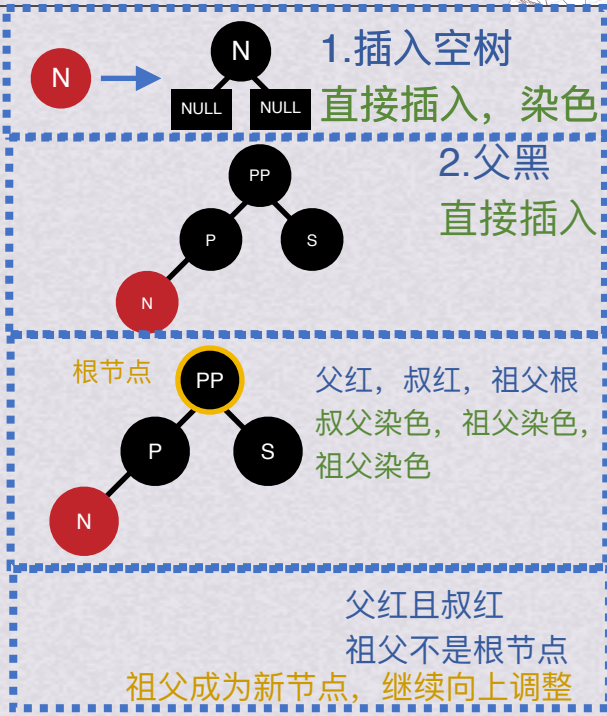
## 4.父红且叔黑/空
## 4-1父为祖父的左孩子

插入节点为其父的左孩子




然后右旋

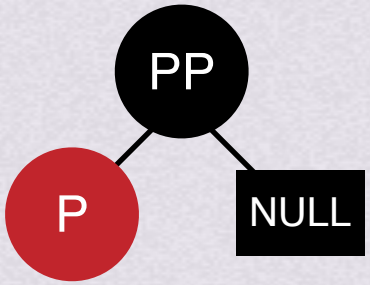
父亲、祖父染色

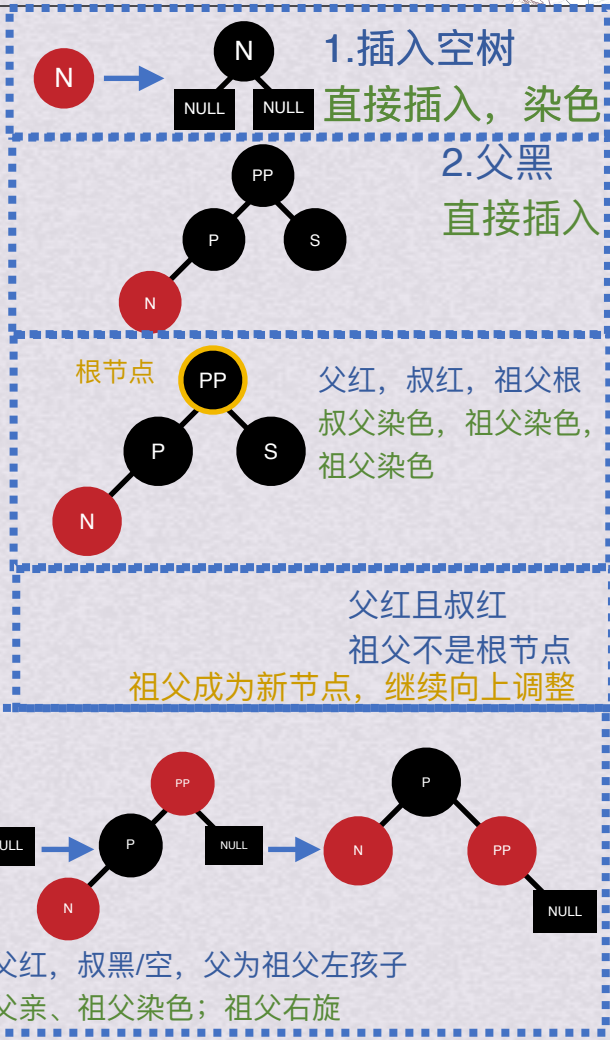


# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的左孩子
插入节点为其父的右孩子

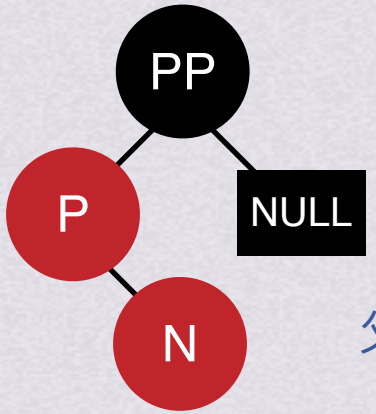


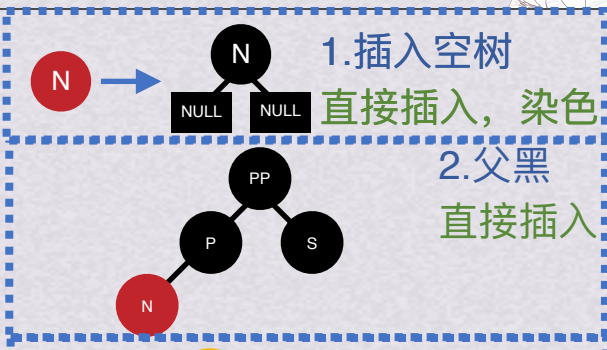
# 红黑树插入

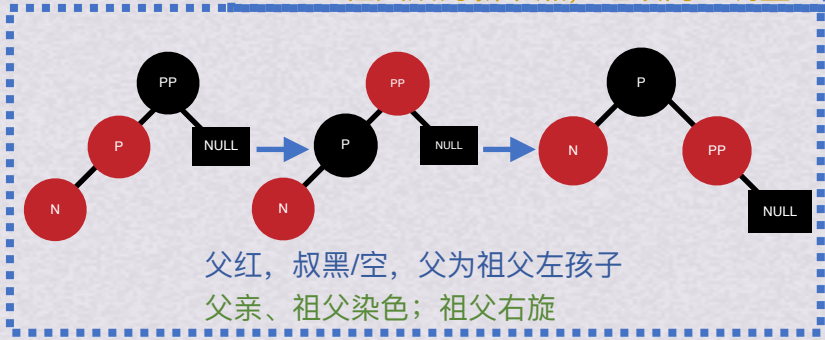
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的左孩子
插入节点为其父的右孩子




父左旋，然后父成新节点



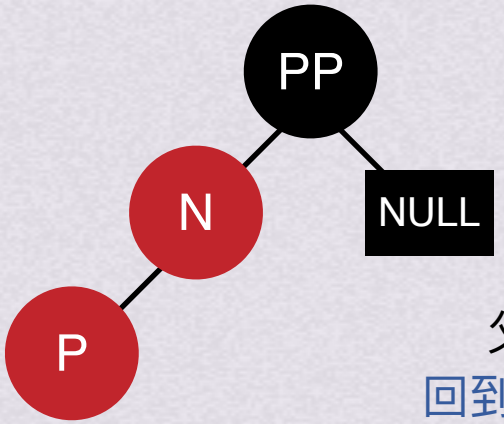



# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

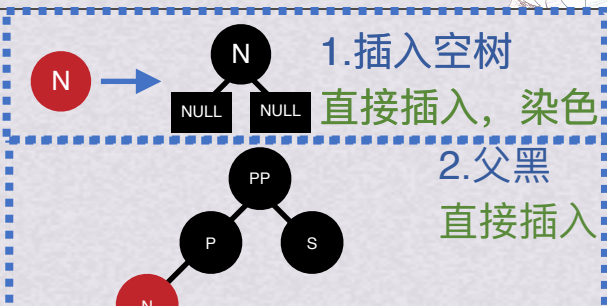
## 4.父红且叔黑/空
## 4-1父为祖父的左孩子

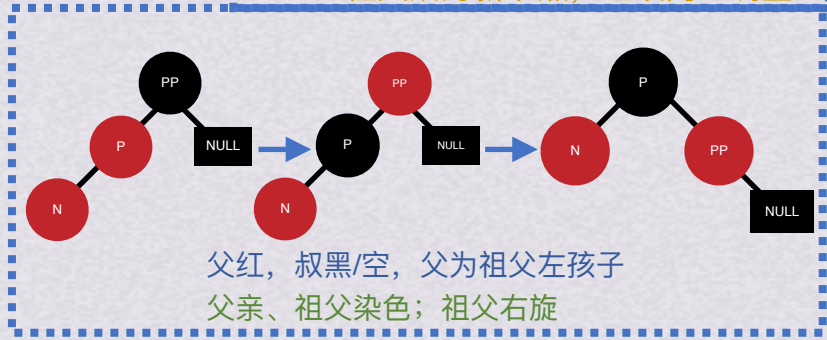
插入节点为其父的右孩子




父左旋，然后父成新节点

回到插入节点为左孩子情况

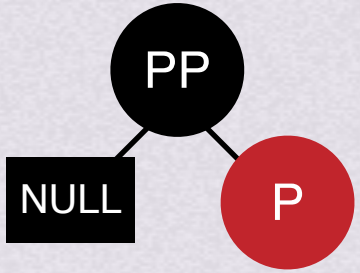


# 红黑树插入

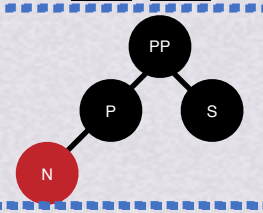
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的右孩子

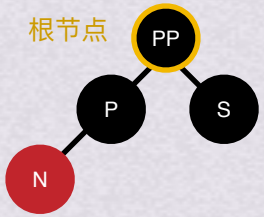




1.插入空树
直接插入，染色

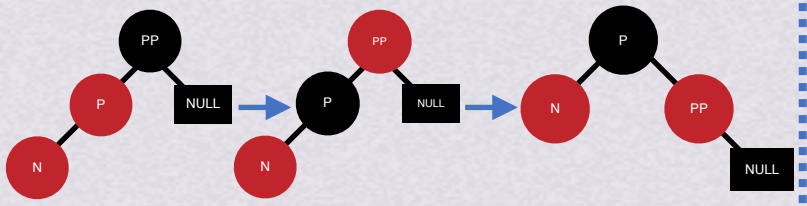



2.父黑
直接插入

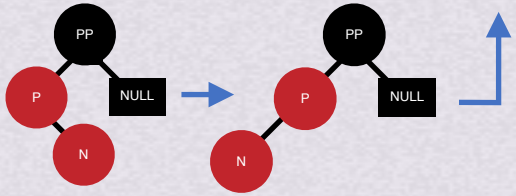



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

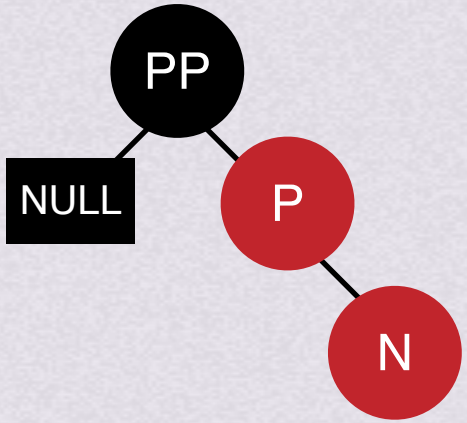



父左旋，然后父成新节点
回到插入节点为左孩子情况

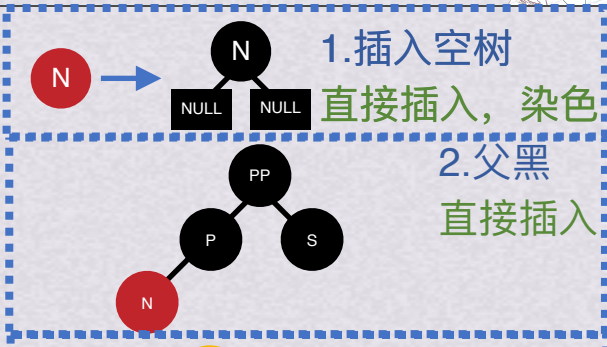
# 红黑树插入

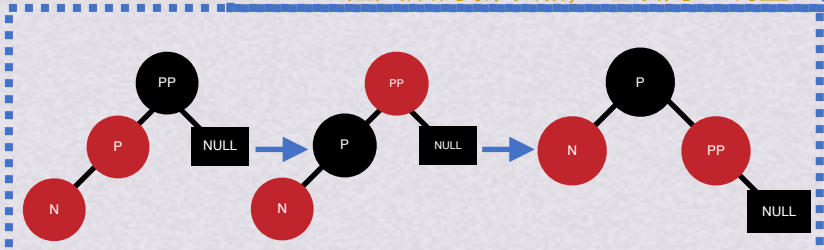
- 插入的节点一定是红节点!
- 染色就是红黑互换!

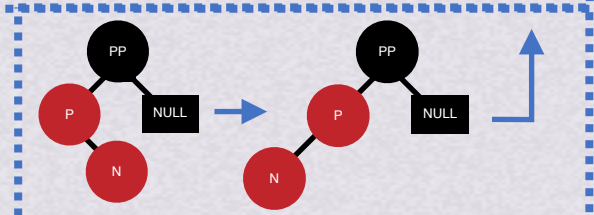
## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的右孩子




父、祖父染色
祖父左旋



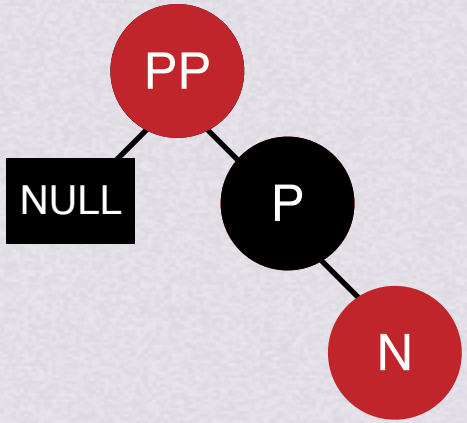



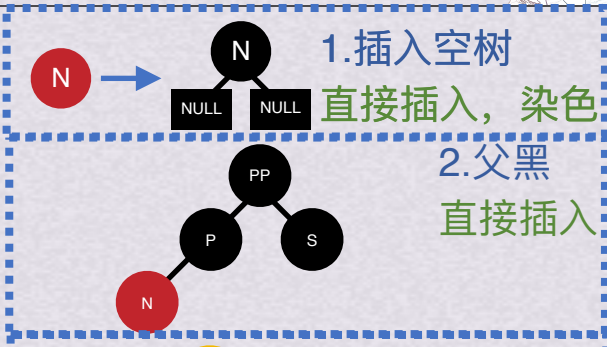
# 红黑树插入

- 插入的节点一定是红节点!
- 染色就是红黑互换!

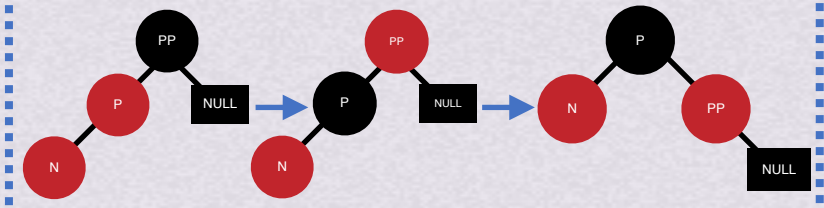
## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的右孩子



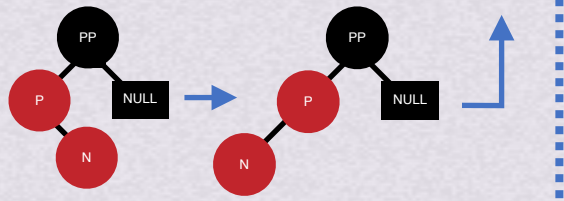

父、祖父染色
祖父左旋




父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

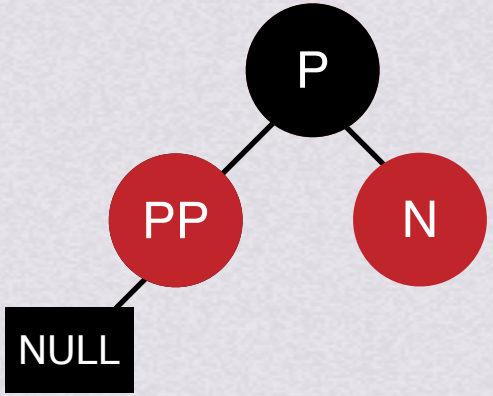



父左旋，然后父成新节点
回到插入节点为左孩子情况

# 红黑树插入

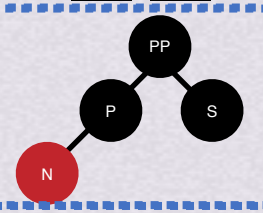
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的右孩子

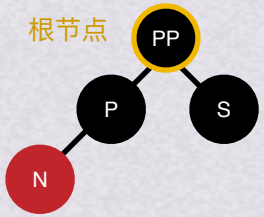



父、祖父染色
祖父左旋
调整完毕


1.插入空树
直接插入，染色

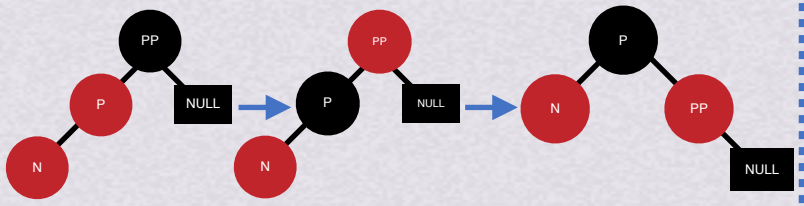



2.父黑
直接插入

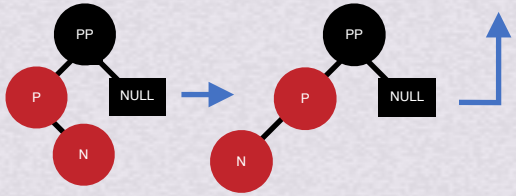



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

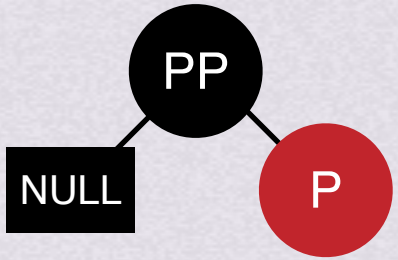



父左旋，然后父成新节点
回到插入节点为左孩子情况

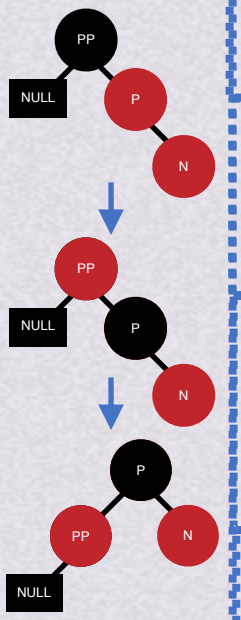
# 红黑树插入

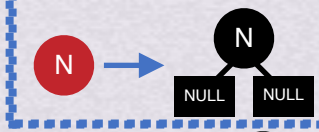
- 插入的节点一定是红节点!
- 染色就是红黑互换!

## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
### 插入节点为其父的左孩子

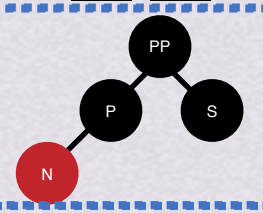



父红，叔黑，插入节点为其父右孩子
父、祖父染色，祖父左旋

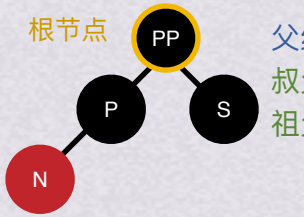



1.插入空树
直接插入，染色

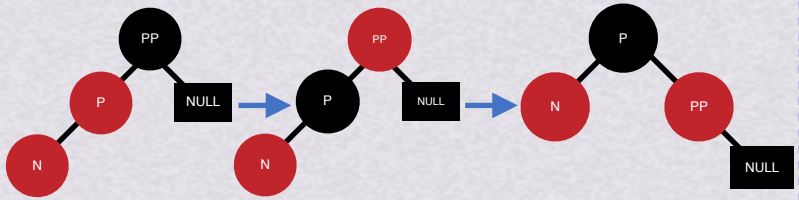



2.父黑
直接插入

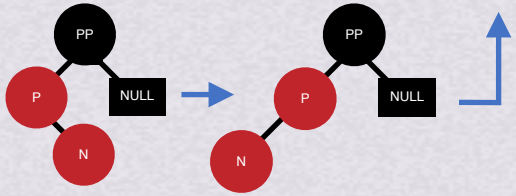



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

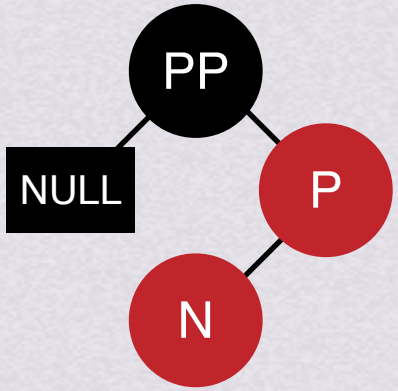



父左旋，然后父成新节点
回到插入节点为左孩子情况

# 红黑树插入

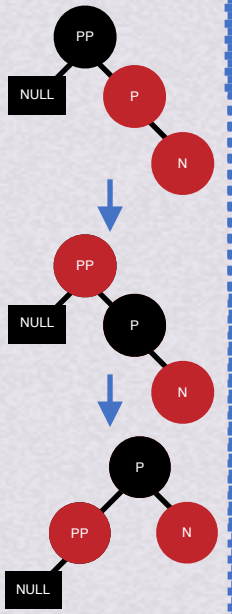
- 插入的节点一定是红节点!
- 染色就是红黑互换!

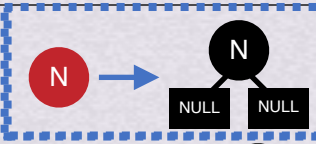
## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的左孩子



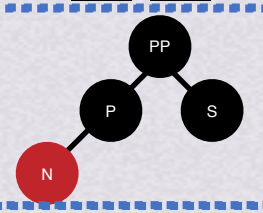

父右旋
父为新节点

父红，叔黑，插入节点为其父右孩子
父、祖父染色，祖父左旋



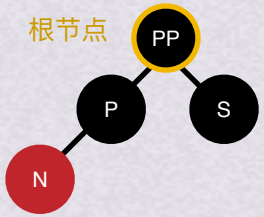




1.插入空树
直接插入，染色



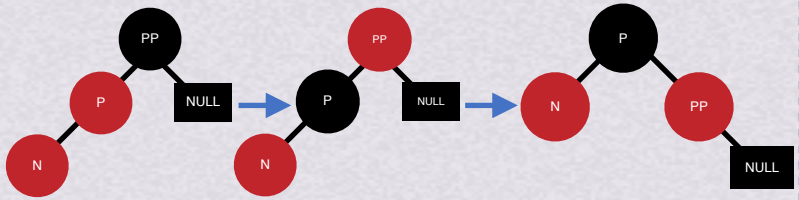

2.父黑
直接插入

根节点

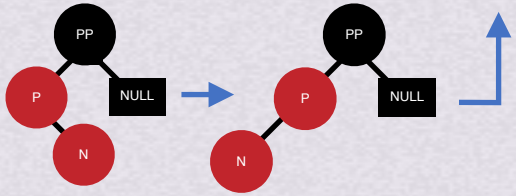



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

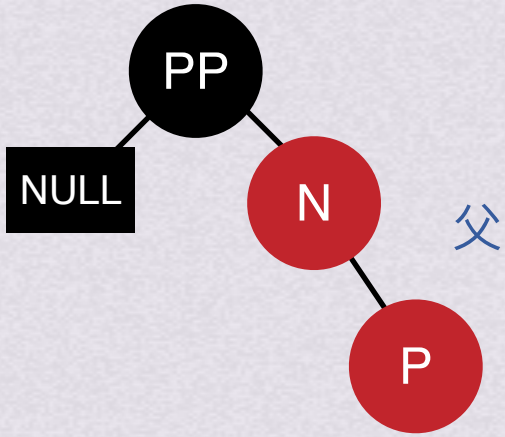



父左旋，然后父成新节点
回到插入节点为左孩子情况

# 红黑树插入

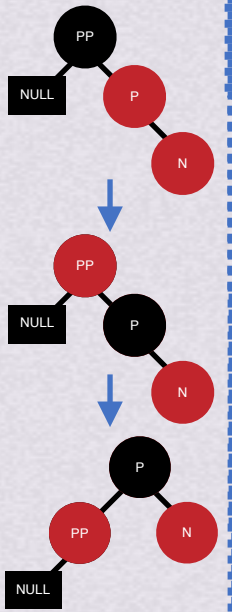
- 插入的节点一定是红节点！
- 染色就是红黑互换！

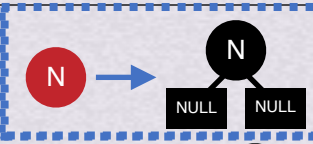
## 4.父红且叔黑/空
## 4-1父为祖父的右孩子
插入节点为其父的左孩子



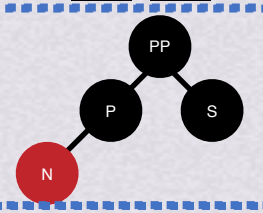

父右旋
父为新节点

父红，叔黑，插入节点为其父右孩子
父、祖父染色，祖父左旋



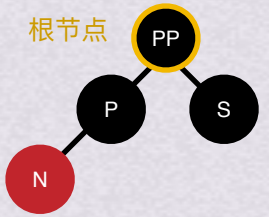




1.插入空树
直接插入，染色



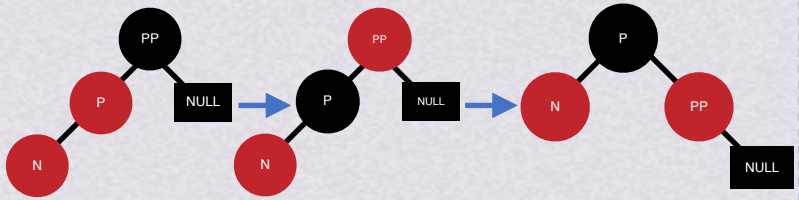

2.父黑
直接插入

根节点

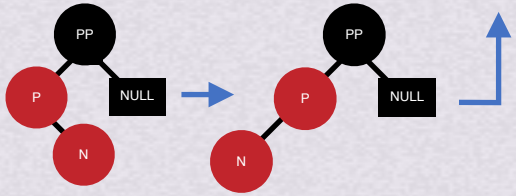



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整




父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

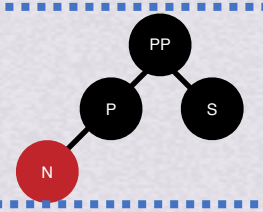



父左旋，然后父成新节点
回到插入节点为左孩子情况

空树


1.插入空树
直接插入，染色

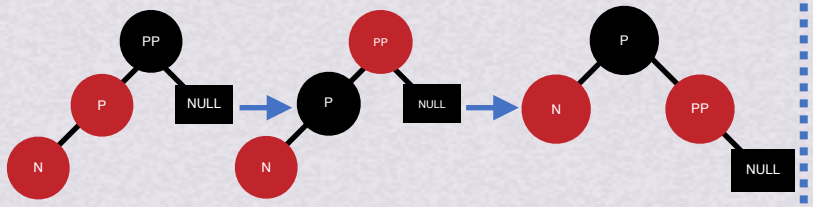
父黑



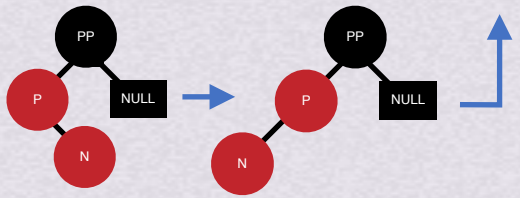

2.父黑
直接插入

父红且叔黑/空

父为祖父左孩子

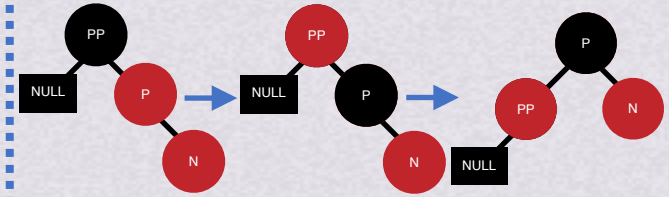



父红，叔黑/空，父为祖父左孩子
父亲、祖父染色；祖父右旋

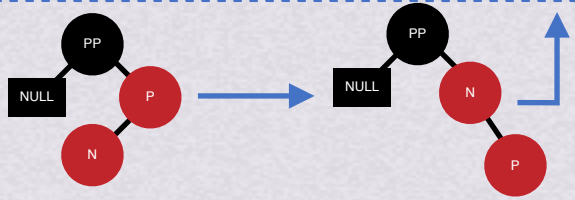



父左旋，然后父成新节点
回到插入节点为左孩子情况

父为祖父右孩子

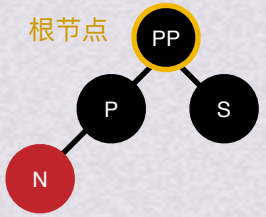



父红，叔黑，插入节点为其父右孩子
父、祖父染色，祖父左旋




父红，叔黑/空，父为祖父右孩子，插入节点为其父左孩子
父亲右旋，成为新节点

父红且叔红

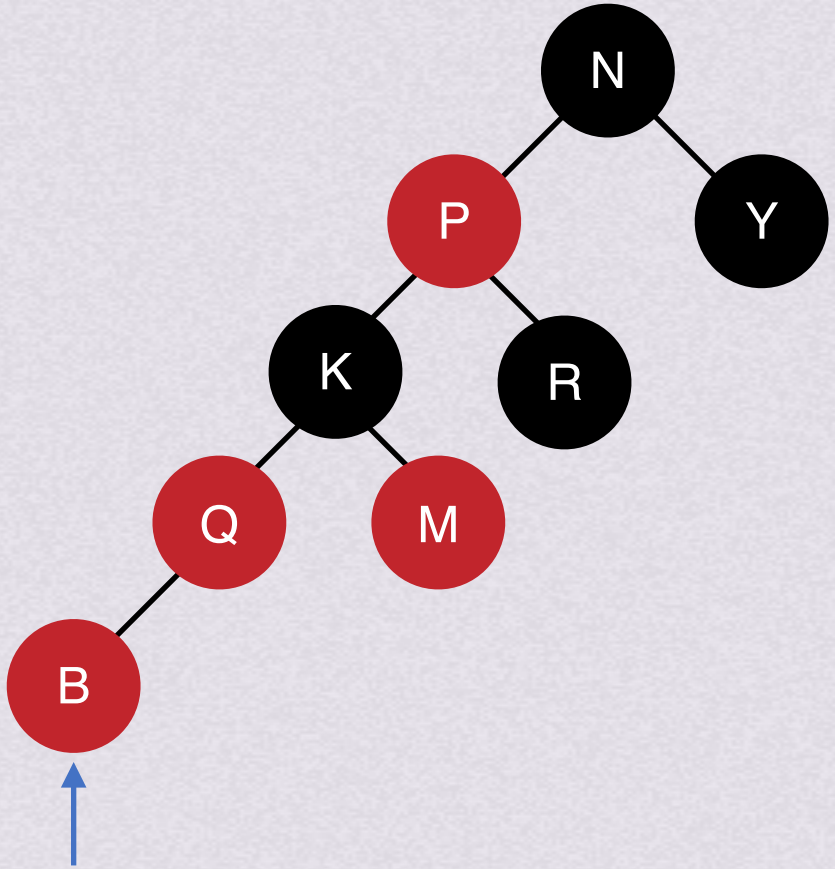



父红，叔红，祖父根
叔父染色，祖父染色，
祖父染色

父红且叔红
祖父不是根节点
祖父成为新节点，继续向上调整

# 红黑树插入

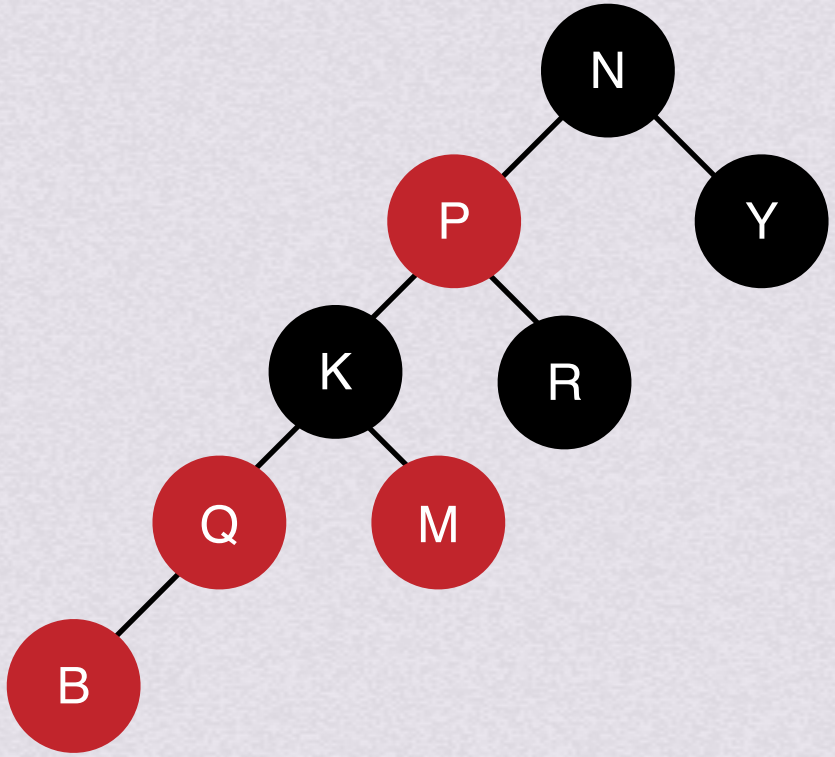
## 小试身手




B为新插入节点，该如何调整?

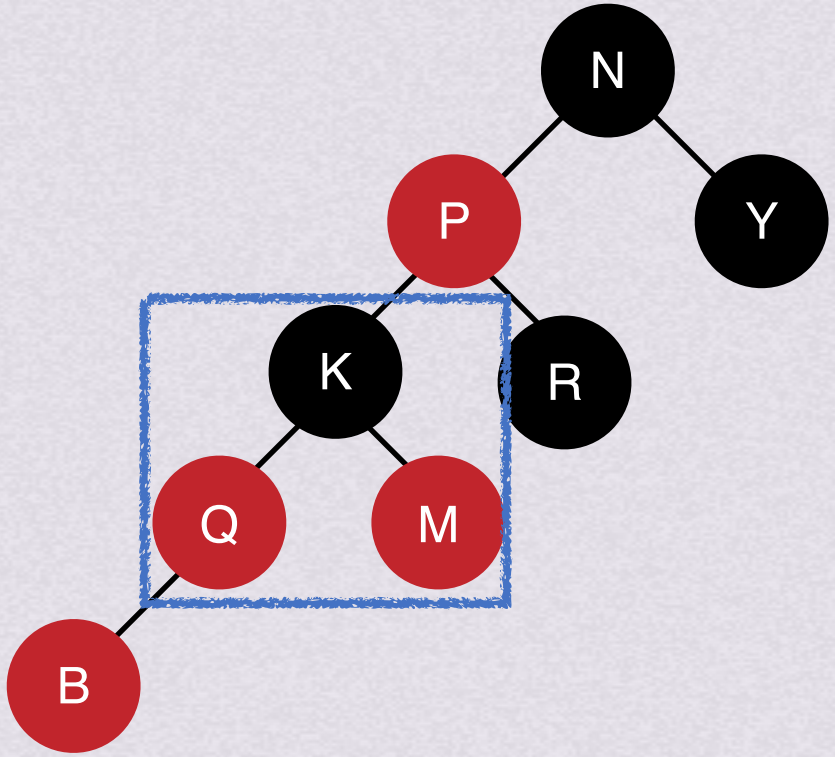
# 红黑树插入

## 小试身手



# 红黑树插入

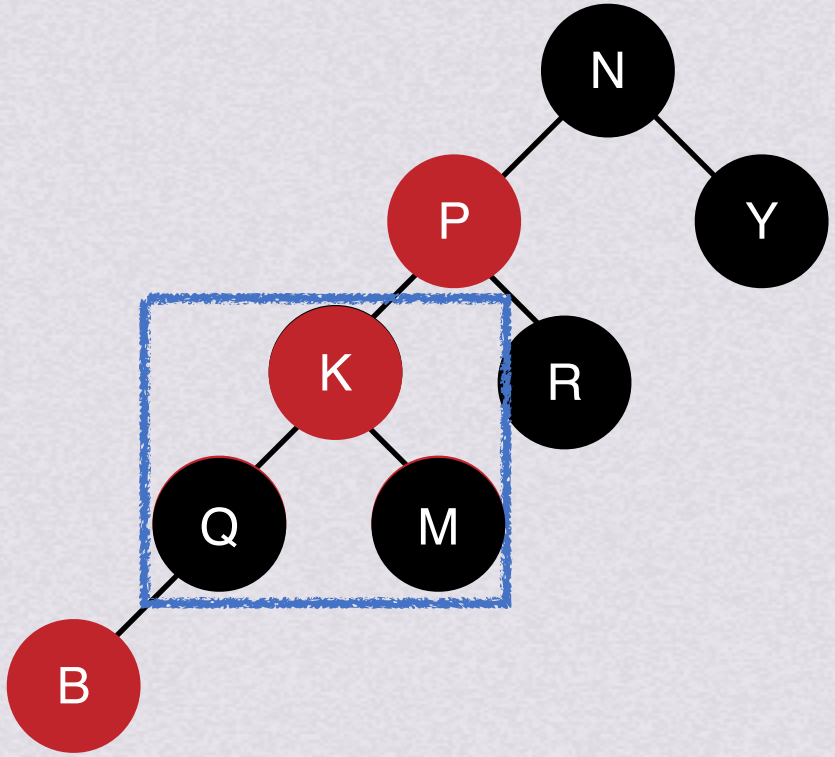
## 小试身手




1.染色父、祖父

# 红黑树插入

## 小试身手



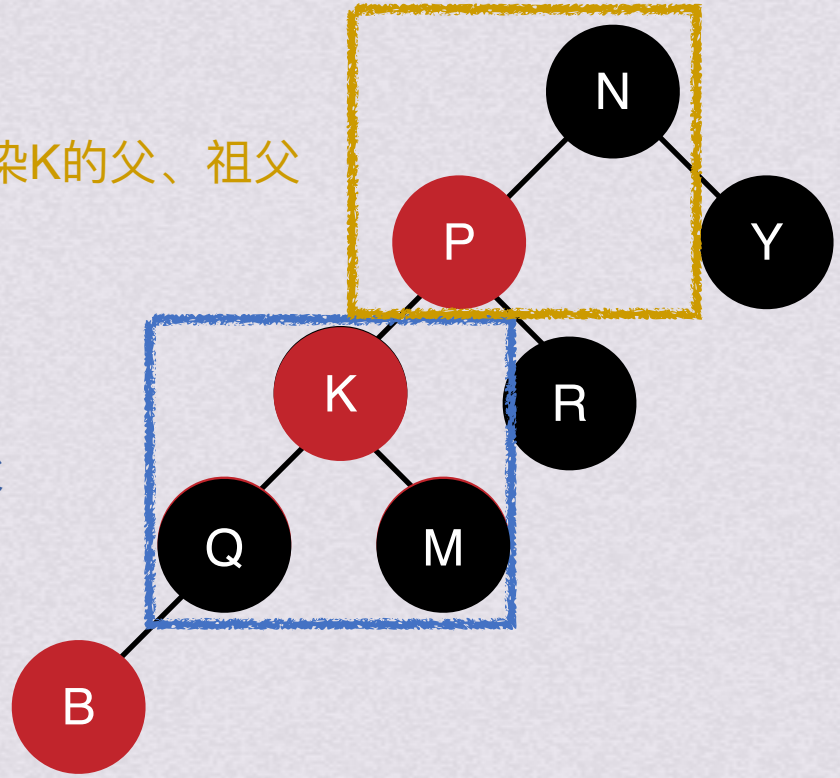

1.染色父、祖父

# 红黑树插入

## 小试身手

2.染K的父、祖父

1.染色父、祖父

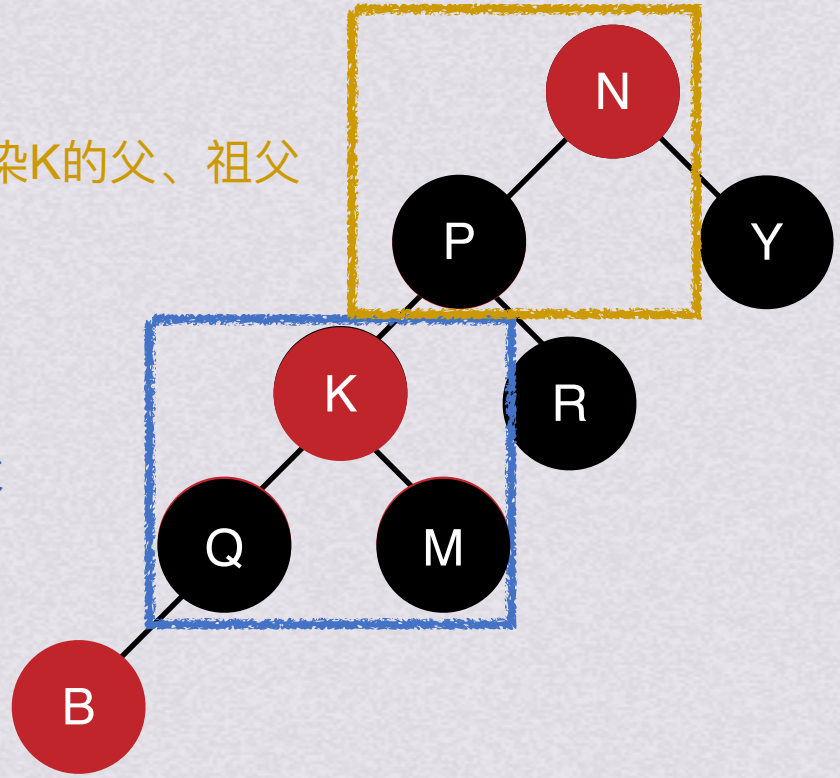


# 红黑树插入

## 小试身手

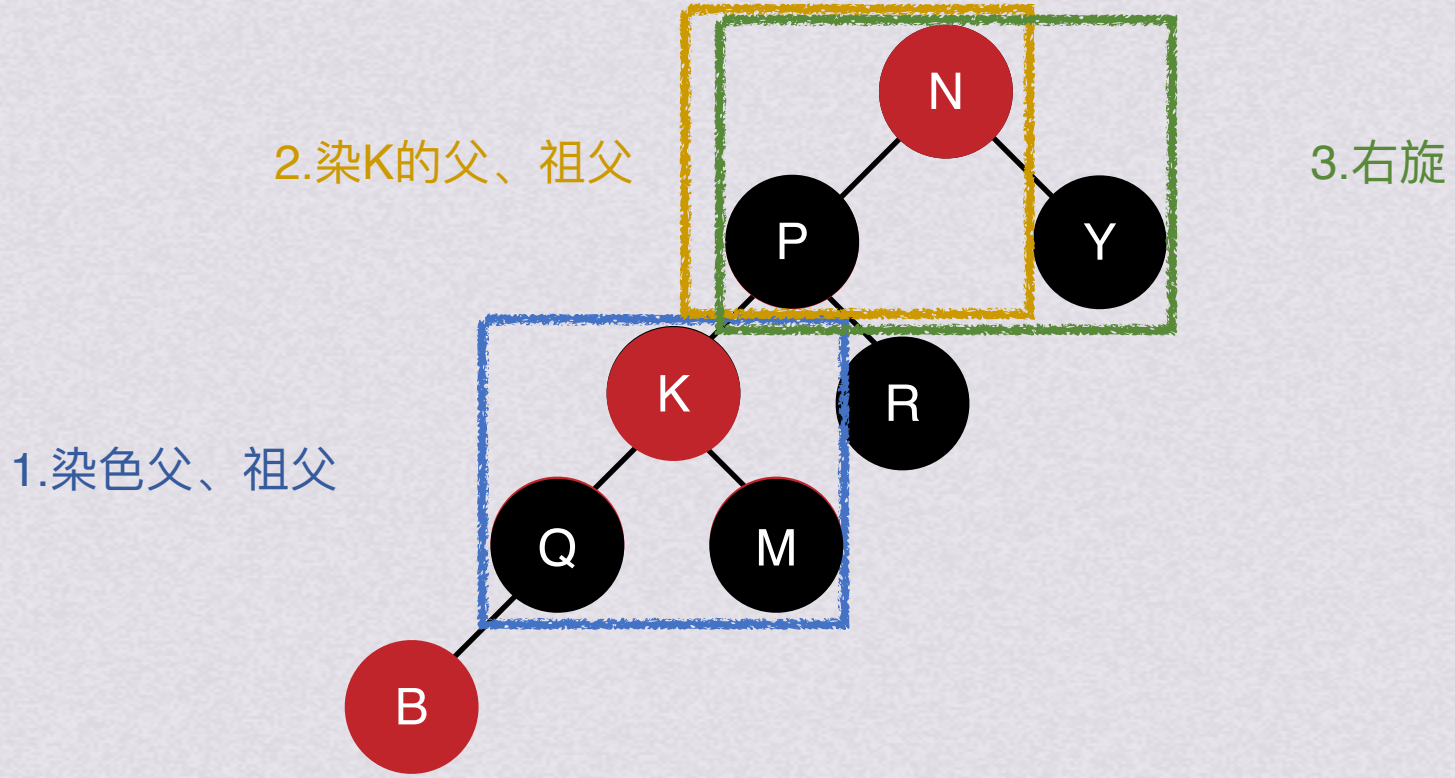
2.染K的父、祖父

1.染色父、祖父



# 红黑树插入

## 小试身手



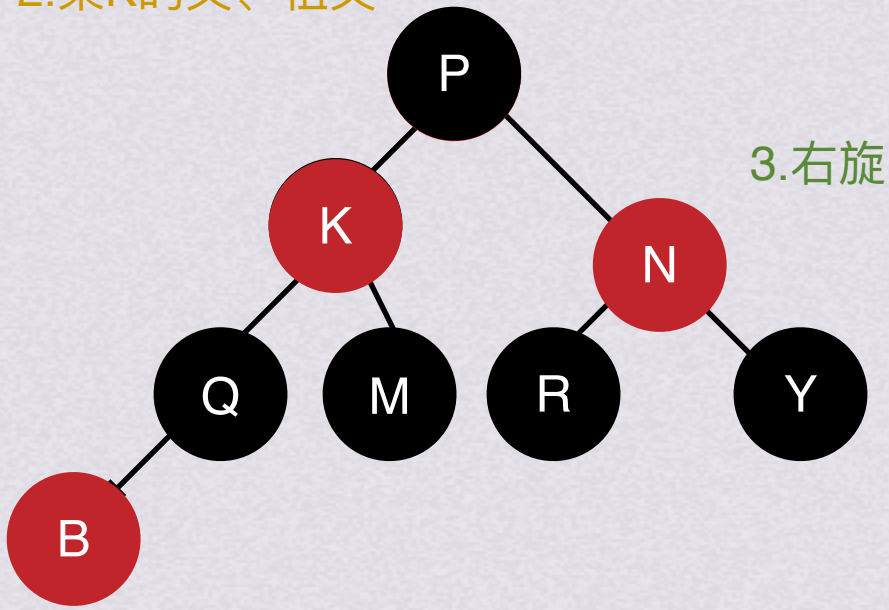
# 红黑树插入

## 小试身手

2.染K的父、祖父

1.染色父、祖父

3.右旋



# 红黑树

## 删除

# 红黑树

## 删除

## 不讲，嘻嘻

# 红黑树删除

| Case # | Check condition | Action |
|---|---|---|
| 1 | If node to be delete is a red leaf node | Just remove it from the tree |
| 2 | If DB node is root | Remove the DB and root node becomes black. |
| 3 | (a) If DB's sibling is black, and<br>(b) DB's sibling's children are black | (a) Remove the DB (if null DB then delete the node and for other nodes remove the DB sign)<br>(b) Make DB's sibling red.<br>(c) If DB's parent is black, make it DB, else make it black |
| 4 | If DB's sibling is red | (a) Swap color DB's parent with DB's sibling<br>(b) Perform rotation at parent node in the direction of DB node<br>(c) Check which case can be applied to this new tree and perform that action |
| 5 | (a) DB's sibling is black<br>(b) DB's sibling's child which is far from DB is black<br>(c) DB's sibling's child which is near to DB is red | (a) Swap color of sibling with sibling's red child<br>(b) Perform rotation at sibling node in direction opposite of DB node<br>(c) Apply case 6 |
| 6 | (a) DB's sibling is black, and<br>(b) DB's sibling's far child is red (remember this node) | (a) Swap color of DB's parent with DB's sibling's color<br>(b) Perform rotation at DB's parent in direction of DB<br>(c) Remove DB sign and make the node normal black node<br>(d) Change colour of DB's sibling's far red child to black. |

When you delete a node, it becomes a NIL node and is marked as a double black, DB, node.